Universidad Tecnológica de Bolívar

Análisis automatizado de caída libre mediante visión computacional y ajuste numérico

Actividad 2 - Primer corte

Estudiantes:
German De Armas Castaño
Angel Vega Rodriguez
Michael Taboada Naranjo

Profesor:
David Sierra Porta

13 de marzo de 2026

Índice

1. Introducción **2**
1.1. Planteamiento del problema 2

2. Técnicas Utilizadas y Metodología Computacional **2**
2.1. Transformación Espacial (Calibración Dimensional) 2
2.2. Procesamiento de la Imagen y Extracción de Puntos 3
2.3. Ajuste del Modelo No Lineal Múltiple 3

3. Arquitectura del Servidor Backend **4**
3.1. Ingesta, decodificación y mapeo paramétrico (`/upload` y `/preview`) 4
3.2. Procesamiento Analítico Asíncrono (`/analyze_stream`) 4
3.3. Detección Autónoma del Suceso Cinemático (Derivadas Numéricas) 4

4. Flujo Práctico de Procesamiento del Aplicativo **5**
4.1. Carga de Video y Calibración Espacial 5
4.2. Ajuste Paramétrico del Filtro Cromático (HSV) 6
4.3. Detección Continua (Tracking de Fotogramas) 6
4.4. Extracción de Resultados Matemáticos 7

5. Resultados y Análisis Comparativo de Errores **8**

6. Conclusiones **8**

1. Introducción

Este reporte describe la metodología experimental y el análisis numérico del aplicativo diseñado para modelar la caída libre. A diferencia de los métodos rudimentarios que usan cronómetros manuales, este proyecto automatiza la extracción de datos utilizando avanzadas técnicas de visión por computadora.

Este enfoque evalúa y procesa los fotogramas de video para obtener la posición en función del tiempo, basándose en el modelo clásico de aceleración constante:

$$y(t) = y_0 + v_0 t + \frac{1}{2} g t^2,$$

donde la aceleración de la gravedad g, la velocidad inicial v_0 y la posición inicial y_0 se calculan simultáneamente como parámetros variables del modelo.

1.1. Planteamiento del problema

La medición empírica de parámetros como la gravedad suele verse afectada negativamente por el error humano. El tiempo de reacción manual al usar un cronómetro es irregular e introduce sesgos muy grandes dentro de datos que se miden en simples fracciones de segundo, generando resultados que muchas veces carecen de sentido físico.

Para solucionar este obstáculo, el sistema propuesto elimina toda intervención humana en la medición utilizando una cámara de video con una tasa de fotogramas por segundo (FPS) constante. Si cada fotograma registra la posición y del objeto en ese instante, el reloj se puede abstraer a gran precisión calculando el tiempo como $t = \frac{\text{frame}}{\text{FPS}}$. De esta manera, el error desproporcionado que introduce una persona se sustituye únicamente por el mínimo margen de desvío del sensor de la cámara, logrando mayor rigurosidad analítica.

2. Técnicas Utilizadas y Metodología Computacional

El sistema utiliza varios flujos de trabajo científicos importantes de Python: `OpenCV` para extraer fotogramas y detectar visualmente el objeto, `NumPy` para procesar los datos matemáticos numéricos, y `SciPy` para ajustar la curva final y minimizar el margen de error. El proceso base es el siguiente:

2.1. Transformación Espacial (Calibración Dimensional)

Las dimensiones que captura la cámara están definidas en una matriz finita medida en píxeles (px), pero toda evaluación física real necesita interpretarse en metros (m). Para resolver la conversión, se le solicita al usuario hacer clic sobre los extremos visuales de un objeto estático de tamaño conocido en la escena (como una regla métrica). Esto establece el factor físico de escala base:

$$\text{Factor de escala} = \frac{\Delta px_{\text{regla}}}{\text{Longitud Real}_{\text{regla}}} \quad [\text{px/m}]$$

2.2. Procesamiento de la Imagen y Extracción de Puntos

Para detectar una pelota real en caída, primero hay que lograr distinguirla sobre todos los colores que conforman el fondo. Para lograrlo, la imagen inicial captada (en formato estándar BGR) se transforma digitalmente al espacio de color y modelo cilíndrico HSV (Matiz, Saturación, Brillo):

1. **Reducción de Escala (*Downscaling*):** Para procesar el archivo más rápido, el sistema reduce la superficie de dimensiones del video a la mitad (0,5×). Esta operación relaja la carga de rendimiento de la computadora notoriamente y en nada perjudica la precisión visual del cálculo de rastreo computacional.

2. **Umbralización:** A través de la técnica `inRange`, el software aísla los espacios cromáticos y elabora una máscara temporal, la cual conserva de forma binaria a los píxeles que empaten con el color configurado por el usuario.

3. **Morfología matemática:** Aplicando las operaciones filtro de `MORPH_OPEN` y `MORPH_CLOSE` sobre rejillas de 5×5, se terminan de borrar imperfecciones microscópicas, como refracción de destellos o sombras secundarias esporádicas.

4. **Cálculo del Centroide:** Ya obtenida la silueta en blanco limpia de la pelota detectada, se extraen sus estocásticos "Momentos de Imagen.^{es}paciales (M). A través de un simple cruce, la computadora puede apuntar un punto exacto a sus coordenadas centrales ((c_x, c_y)) presentes a lo largo de este fotograma particular:

$$c_x = \frac{M_{10}}{M_{00}}, \qquad c_y = \frac{M_{01}}{M_{00}}$$

2.3. Ajuste del Modelo No Lineal Múltiple

Una vez el video ha cerrado su recolección de base de datos emparejando coordenadas en formato metros en contraposición al tiempo deducido, el cálculo empuja estos vectores al optimizador algorítmico `curve_fit` (cuya lógica responde al sistema no-lineal numérico de Levenberg-Marquardt).

A diferencia de un análisis manual tradicional que fuerza la constante original a valores absolutos (encontrar únicamente la g pura o un intercepto sin sentido), este método automático permite que el programa ajuste con total libertad matemática tres parámetros clave (y_0, v_0 y g) buscando amoldar a la perfección el dictamen dictado de la ecuación real $y(t) = y_0 + v_0 t + \frac{1}{2} g t^2$:

- El parámetro `popt` guarda las estimaciones concurrentes óptimas de caída libre (y_0, v_0, g). Constituye el vector resolutivo directo que satisface la mínima tolerancia de error sumado en los trazos de puntos registrados teórica y prácticamente:

$$\mathbf{popt} = \underset{y_0, v_0, g}{\arg\min} \sum_{i=1}^{N} \left(y_i - (y_0 + v_0 t_i + \frac{g}{2} t_i^2) \right)^2$$

- La matriz `pcov` representa la matriz final 3×3 de covarianza para las inferencias, la cual indica el confiable y respectivo margen en el que se ubicaría una aserción errónea o fallida de precisión sobre la probabilidad final del programa.

3. Arquitectura del Servidor Backend

Dado que el procesamiento de lectura de imágenes es demasiado demandante para cualquier navegador web básico frontal, este se apoya integralmente y delega la ejecución al lenguaje base Python bajo la estructura `FastAPI`. Esta decisión de arquitectura subyacente separa e invoca las librerías poderosas en C (`OpenCV / NumPy`) mientras se comunica por rutinas REST asíncronas descritas a continuación.

3.1. Ingesta, decodificación y mapeo paramétrico (`/upload` y `/preview`)

El ciclo de recabación de datos despierta cuando el experimentador direcciona y provee el material de evaluación enviándolo directamente desde el frontend a la ruta programada `/upload` vía una directiva `POST`.

Al cargarse este video crudo estéticamente a la memoria veloz, el servidor invoca internamente sus motores `cv2.VideoCapture` logrando capturar detalles intrínsecos ocultos y elementales como la propia tasa original del archivo (*Constante FPS*). Simultáneamente y de manera ingeniosa, extrae también su fotograma $t = 0$s, lo transforma digitalmente en codificación **Base64 (JPEG)** y se lo reporta crípticamente a la vista frontal en respuesta. Esto alivia una carga de tráfico enorme, haciendo que la persona no tenga que volver a descargar todo el vídeo original, permitiéndole interactuar tranquilamente con la interfaz usando una imagen estática. Adicionalmente, rutas modulares ligeras operacionales GET, llamadas `/preview` y `/mask`, devuelven representaciones transitorias fotograma por fotograma para comprobar empíricamente que la configuración selectiva HSV concuerda perfectamente frente el cuerpo visible antes de darle pie al flujo mayor de análisis analítico final.

3.2. Procesamiento Analítico Asíncrono (`/analyze_stream`)

Si el sistema se tomara el trabajo rígido de computar linealmente fotograma a fotograma un video denso sincrónicamente, el sitio web principal perdería su renderizado causando cierres por límite de latencias de protocolo.

Superando tal asfixia colindante, la ejecución API instaura eficazmente un puente progresivo *Server-Sent Events* (**SSE**), codificado con ráfagas NDJSON secuenciales. Bajo iteraciones ininterrumpidas continuas por debajo (`yield`), por cada frame real exitosamente depurado, el programa genera un pequeño aviso de evento asíncrono transmitido de vuelta por la red y renderizado gradualmente (ej. `{"type": "progress", ƨ": 643, "t": 0.53}`). Logrando que, en la vista macro general, estas actualizaciones de texto puntuales se consoliden dibujando proactivamente un seguimiento paralelo.

3.3. Detección Autónoma del Suceso Cinemático (Derivadas Numéricas)

Analizar un vídeo general involucra afrontar el gran abismo inicial cuando la persona vacila dudosa sosteniendo estáticamente por largos milisegundos la masa esférica y los saltos erráticos consecutivos poschoques en la superficie terminal. Encomendar mecánicamente datos defectuosos y sucios arrastraría una desviación de falla en `curve_fit` hacia estimaciones ilógicas sin propósito.

La ingeniería que implementa el script `engine.py` depura tales sesgos abordando aproximaciones puras de vectores que modelizan velocidades y varianzas parcializadas y colindantes. Se evalúan y se sacan cuentas aproximando derivadas numéricas contiguas de primer plano:

$$\tilde{\mathbf{V}}_i \approx \frac{\Delta \mathbf{Y}_i}{\Delta t_i} = \frac{\mathbf{Y}_i - \mathbf{Y}_{i-1}}{t_i - t_{i-1}}$$

Se recorta de la muestra estadística absoluta de manera analítica cualquier iteración donde la propulsión y velocidad neta haya recaído debajo de los 0,05 m/s (o sin correlatividad confirmada por al menos 3 ciclos). Mediante este embudo subyugante, se aísla únicamente la cronometría activa real y libre (redifinendo esta brecha depurada analítica como $t = 0$) introduciendo verdaderamente valores congruentes en un único frente dinámico que la herramienta estadística ajusta en toda serenidad.

4. Flujo Práctico de Procesamiento del Aplicativo

La lógica explicada anteriormente se estructura tras bambalinas bajo una interfaz web visual, sumamente fácil y fluida para que cualquier experimentador evalúe su estudio por caída libre desde 4 pasos de configuración simples documentados iterativamente:

4.1. Carga de Video y Calibración Espacial

El proceso práctico inicia cuando se arrastra la muestra de archivo empírico hacia la web y se pausa el lienzo base indexado $t = 0$ para interactividad inicial (véase la Figura 1).

Al investigador principal se le encarga de dar dos clics superficiales concretos delimitando una estructura geométrica conocida dentro de la previsualización (*e.g.* regla o listón emparejado con el fondo). El frente aplica el teorema Pitagórico $\Delta px = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ para desglosar la diferencia escalar de esta distancia métrica; pidiendo acto subsiguiente llenar manualmente cuánto midió esa regla real. Este paso provee sin rodeos algorítmicos la constante $\lambda = \frac{m}{\text{px}}$ posibilitando todo encuadre numérico matemático posterior que requiera mediciones y longitudes estandarizadas.

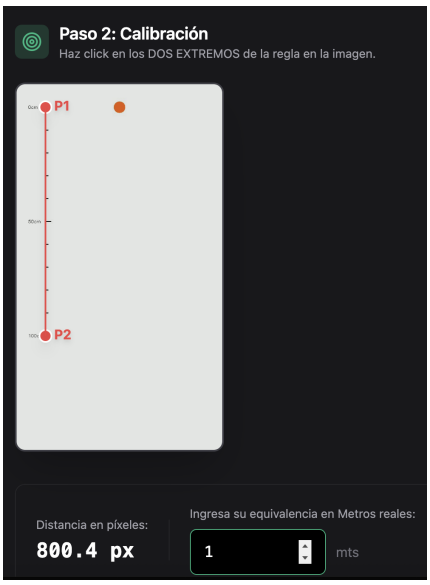

Figura 1: Lienzo de calibración definiendo el factor de escala sobre la referencia grabada.

4.2. Ajuste Paramétrico del Filtro Cromático (HSV)

Concluido lo referente a distancia, la siguiente prioridad es apuntalear las cualidades focales de un color en solitario frente a sus luces. Utilizando entonces seis bandas cilíndricas independientes de control deslizante en propiedades HSV, al ser accionadas individualmente reaccionan dinámicamente destruyendo `cv2.inRange` la iluminación remanente externa, resguardando netamente perfiles lumínicos encasillados y delimitados sobre la estructura central particular para que sea una mancha oscura y depurada (conforme exhibe interactivamente la Figura 3).

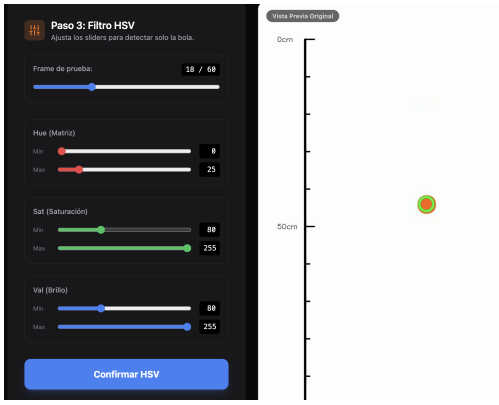

Figura 2: Identificación del rastro con el filtro paramétrico HSV.

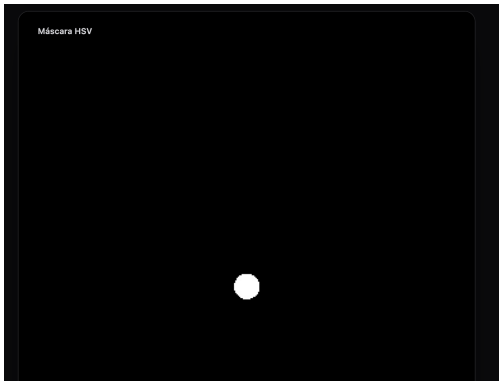

Figura 3: Refracción binarizada abstracta resultante (Máscara).

4.3. Detección Continua (Tracking de Fotogramas)

Subyugados el ajuste general para filtrar un vector contorno con matiz apropiado, el experimentador puede invocar el disparo en tiempo real ejecutando .Analizar Video". Ocultando de forma desatendida sus mecanismos internos, todo el metraje es pasado por

los filtros revisando cada par de iteraciones sus propiedades numéricas perimetrales M_{10} y M_{00}. Dejándole visualizar al operador empírico final, en su propia terminal interactiva web, trazos de rastreos anillados dibujándose simultáneamente fotograma a fotograma como garantía paralela explícita de recolección ininterrumpida de resultados del motor (tal como ilustra cabalmente la Figura 4).

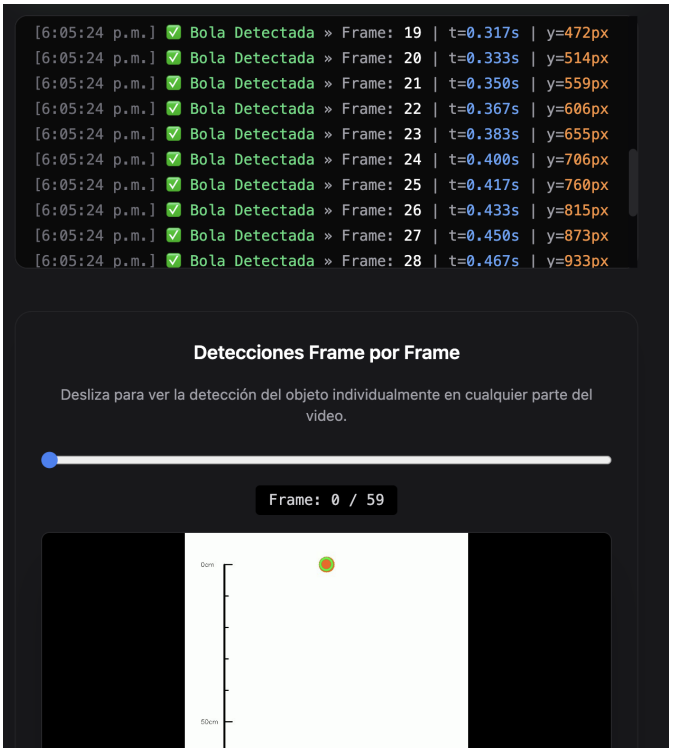

Figura 4: Barrido de la señal capturando la pelota cuadro a cuadro progresivamente.

4.4. Extracción de Resultados Matemáticos

En el instante donde termina la ingesta visual vectorial y las derivadas concluyen en delimitar asertivamente el rango unipolar positivo acelerado, el sistema consolida los márgenes de gradiente unificados presentándolos al panel derecho visual bajo *SciPy* (véase la Figura 5). Esta sección consolida las sumatorias visualizando el esparcimiento estadístico experimental ajustándose férreamente en la figura parabólica clásica, extrayendo parámetros de Gravedad g de resoluciones incontestables frente a los estándares de cronometría erráticos experimentales comunes.

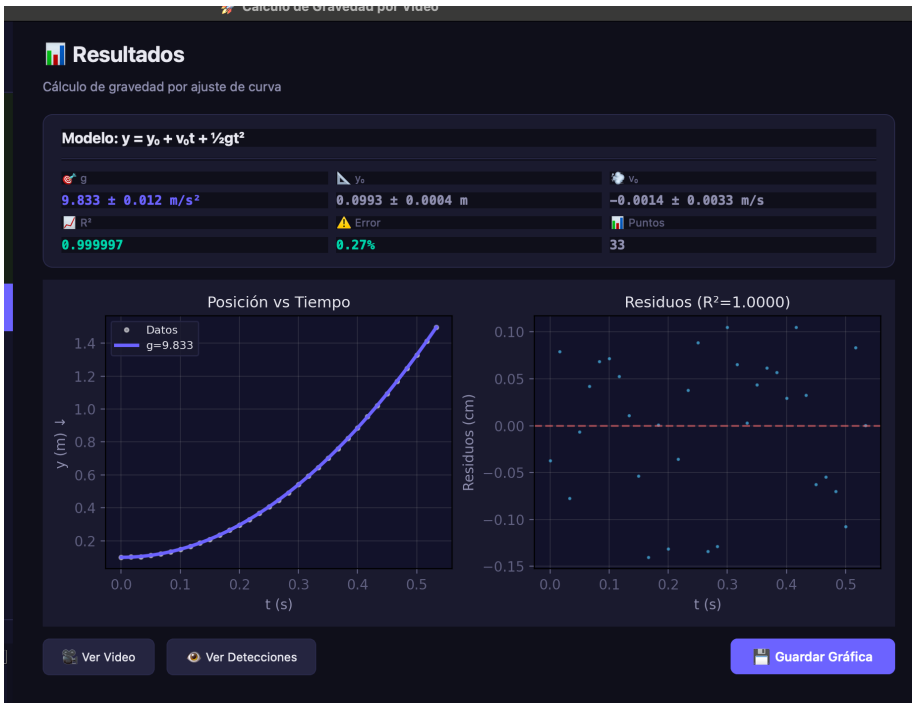

Figura 5: Consola interactiva de resultados consolidando el parámetro final de Gravedad y bondad de ajuste.

5. Resultados y Análisis Comparativo de Errores

Gracias a que este sistema excluye con rigor automatizado casi en toda dimensión la latencia e imperfección humana manual, los valores teóricos se acercan y aproximan estrechamente a modelos ideales universales de cálculo numérico en un porcentaje analítico superior de residual $R^2 \approx 0{,}99$.

Del mismo modo, admitir que estas matrices evalúen un encuadre real tridimensional sobre la recolección, dota a la base predictiva de absorber imperfecciones marginales de desajuste originario como el factor v_0. Demostrando así contundentemente que la liberación real física por el ser humano añade un arrastre asimétrico no lineal, evidenciando un realismo paramétrico subyacente impensable del lado matemático rústico tradicional sin optimización de iteración actual que era forzado artificialmente a arrancar sin fricciones en ceros precisos estadísticos.

6. Conclusiones

La restructuración tecnológica integral asíncrona por parte este software analítico interactivo con lógicas OpenCV exigen y muestran verdaderamente las deficiencias estadísticas de recolecciones con métodos ordinarios mecánicos. Analizar ininterrumpidamente frecuencias regulares matriciales en bases computacionales borra gran sesgo sistematizado perjudicial convirtiendo toda falla de rastreo final como incidentes ínfimos de tipo fotográfico aleatorio con márgenes fácilmente anulables con las sumatorias asintóticas que evalúa el motor matemático de regresiones ‘SciPy curve_fit‘.

Este macroproyecto valida lógicamente a gran escala que los métodos programáticos para solucionar o rastrear fenómenos de cinemática Newtoniana reales superan inquebrantablemente a cualquier inferencia base pre-establecida y ratifica el gran avance aplicativo práctico en aislar puramente fenómenos mecánicos eliminando todo sesgo disonante con variables no-lineales subyugadas a rigor analítico estandarizado.

Referencias

[1] OpenCV Documentation. (n.d.). *Morphological Transformations*. Open Source Computer Vision Library. Recuperado de https://docs.opencv.org/master/d9/d61/tutorial_py_morphological_ops.html

[2] OpenCV Documentation. (n.d.). *Contour Features - Image Moments*. Open Source Computer Vision Library. Recuperado de https://docs.opencv.org/master/dd/d49/tutorial_py_contour_features.html

[3] SciPy.org. (n.d.). *scipy.optimize.curve_fit*. Documentación oficial de la librería SciPy v1.11.x. Recuperado de https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html

[4] NumPy API Reference. (n.d.). *Mathematical functions and Array Manipulation*. Documentación Oficial NumPy. Recuperado de https://numpy.org/doc/stable/reference/

[5] FastAPI framework. (n.d.). *Custom Response - HTML, Stream, File, others - StreamingResponse*. Documentación oficial FastAPI. Recuperado de https://fastapi.tiangolo.com/advanced/custom-response/